innocent-looking calls are genuine external service calls to a real server that exists in production but not in the test:

```
def process_flit
  # a bunch of messy stuff
  flit_server.check_for_mentions(self)
  # more messy stuff
  flit_server.follower_list(user)
  # more messy stuff
end
```

Now you need to get the process_flit method under test. The test might look like:

```
it "processes a flit correctly if it has followers" do
  user = User.create(screen_name: "zot")
  follower = user.followers.create(screen_name: "jennyw")
  flit = Flit.new(user, "Hello to $jennyw, How are things on earth?")
  flit.process_flit
  expect(follower.timeline.size).to eq(1)
end
```

For this test to work, you need to prevent the flit_server object in the original code from calling the production server that will not exist in the test environment. For the sake of argument, let's assume there's a compelling reason a normal test-double package can't be used here—possibly because the flit_server object is too tightly intertwined with the rest of the code. You have two problems to solve: you need to create a flit_server object that will perform test-safe activities when called, and you need to inject that object into the test so it's used when the method is run under test.

Luckily, Ruby is extremely flexible when it comes to redirecting code execution. (Of course, this is exactly the kind of flexibility that drives security-minded programmers from other languages crazy. But here you're using for it good, not evil.) In a compiled object-oriented language, you might have to create a new subclass of the expected object and override the methods in question. You can do that in Ruby, too:

```
class TestFlitServer < FlitServer
  def check_for_mentions(flit)
    # test code
  end

  def follower_list(user)
    # test code
  end
end
```

Depending on the details of the FlitServer class, you may have to override other methods, such as the constructor.